

Week 7-8: AES-GCM Nonce Reuse Attack & Mitigation

1. Context, Risks, and Security Goals

1.1. Context

AES-GCM (Galois/Counter Mode) is a widely adopted Authenticated Encryption standard due to its performance and parallelizability. However, its security model is extremely brittle regarding the Initialization Vector (IV) or Nonce.

In this experiment, we analyze a "Forbidden Attack" scenario where a server initializes the AES-GCM cipher using a static, globally defined Nonce for multiple encryption requests. This mimics real-world errors found in embedded firmware or poorly designed APIs where state is improperly managed.

1.2. Risks Assessment

- **Authentication Key Recovery:** Reusing a nonce allows an attacker to XOR two valid authentication tags, eliminating the encryption mask and isolating the GHASH polynomial. This allows algebraic recovery of the authentication key H .
- **Universal Forgery:** Once H is recovered, an attacker can generate valid Authentication Tags for *any* ciphertext. In our specific case, the attacker can forge a valid tag for the command give me the flag, which the server would otherwise reject.

1.3. Security Goals

- **Uniqueness:** The critical security goal for GCM is that the pair (key, nonce) must **never** repeat.
- **Integrity:** The system must ensure that no entity without the secret key can generate a valid Tag for a modified ciphertext.

2. Solution Architecture

To demonstrate this vulnerability, we designed a Client-Server architecture based on the provided source code:

2.1. The Vulnerable Server

- **Service:** A Python script exposing a menu-driven interface (1. Encrypt, 2. Decrypt) over a socket or standard I/O.
- **Configuration:**
 - **Algorithm:** `AES.new(KEY, AES.MODE_GCM, nonce=NONCE)`.
 - **The Flaw:** The NONCE variable is generated once at startup (`NONCE = os.urandom(16)`) and reused for every subsequent encryption call in the `encrypt()` function.
- **Access Control:** The server explicitly refuses to output the tag if the plaintext contains "flag", forcing the attacker to forge it.

Here is the main core in script:

```
def encrypt():
    plaintext = bytes.fromhex(input("Enter the plaintext: "))
    cipher = AES.new(KEY, AES.MODE_GCM, nonce=NONCE)
    ciphertext, tag = cipher.encrypt_and_digest(plaintext)
    if b"flag" in plaintext:
        print("Error: Invalid plaintext, not authenticating", flush=True)
        print("Ciphertext:", ciphertext.hex(), flush=True)
    else:
        print("Nonce:", NONCE.hex(), flush=True)
        print("Ciphertext:", ciphertext.hex(), flush=True)
        print("Tag:", tag.hex(), flush=True)

def decrypt():
    c_hex = input("Enter the ciphertext: ")
    t_hex = input("and the tag: ")
    ct = bytes.fromhex(c_hex)
    tag = bytes.fromhex(t_hex)
    cipher = AES.new(KEY, AES.MODE_GCM, nonce=NONCE)
    try:
        decrypted = cipher.decrypt_and_verify(ct, tag)
    except ValueError as e:
        print("Error: Invalid authentication tag", flush=True)
        return

    if b"give me the flag" in decrypted:
        print("Plaintext:", flag.hex(), flush=True)
    else:
        print("Plaintext:", decrypted.hex(), flush=True)
```

2.2. The Attacker

- **Tools:** Python pwntools for connection management and **SageMath** for algebraic computations over Galois Fields ($\text{GF}(2^{128})$).
- **Attack Logic:**
 1. **Collection:** Request encryption of two known payloads (payload1, payload2) to obtain ciphertext/tag pairs (C_1, T_1) and (C_2, T_2) .
 2. **Polynomial Solving:** Construct a polynomial $P(x)$ derived from:
$$T_1 \oplus T_2 = \text{GHASH}_H(C_1) \oplus \text{GHASH}_H(C_2)$$
 3. **Root Finding:** Use SageMath to find the roots of $P(x)$ in $\text{GF}(2^{128})$. One of these roots is the authentication key H .
 4. **Forgery:** Use H to compute the correct tag for the target payload give me the flag and authenticate to the server.

Here is the main attack core:

```
io = remote("localhost", 21337)

payload1 = b"A" * 32
payload2 = b"B" * 32
payload3 = pad(b"give me the flag", 32)

C1_result = enc_msg(payload1)
C1 = convert_to_blocks(C1_result[0])
T1 = C1_result[1]

C2_result = enc_msg(payload2)
C2 = convert_to_blocks(C2_result[0])
T2 = C2_result[1]

C3_result = enc_msg(payload3)
C3 = convert_to_blocks(C3_result[0])

L = struct.pack(">QQ", 0 * 8, len(C1) * 8)
C1_p = [bytes_to_polynomial(C1[i], a) for i in range(len(C1))]
C2_p = [bytes_to_polynomial(C2[i], a) for i in range(len(C2))]
C3_p = [bytes_to_polynomial(C3[i], a) for i in range(len(C3))]
T1_p = bytes_to_polynomial(T1, a)
T2_p = bytes_to_polynomial(T2, a)
L_p = bytes_to_polynomial(L, a)

G_1 = (C1_p[0] * x**3) + (C1_p[1] * x**2) + (L_p * x) + T1_p
G_2 = (C2_p[0] * x**3) + (C2_p[1] * x**2) + (L_p * x) + T2_p
G_3 = (C3_p[0] * x**3) + (C3_p[1] * x**2) + (L_p * x)
P = G_1 + G_2
auth_keys = [r for r, _ in P.roots()]

for H, _ in P.roots():
    EJ = G_1(H)
    T3 = G_3(H) + EJ
    calc_tag = polynomial_to_bytes(T3)
    try:
        plaintext = dec_msg(C3[0] + C3[1], calc_tag)
        print(f"\nPLAINTEXT: {plaintext.decode()}")
        break
    except:
        print("Decryption failed")
        continue
```

3. Implementation and Evaluation

3.1. Experimental Environment Setup

The environment was set up to simulate a service that statically configures its crypto parameters (a common mistake in embedded firmware or poor API design).

Component	Role	Technical Configuration
Server	Vulnerable Target	Python script; AES-GCM; Static Nonce Intentionally suppresses tags for "flag" requests to simulate access control.
Attack Script	Attacker Tooling	Python + SageMath: Uses polynomial rings over $GF(2^{128})$ to solve for the Auth Key.
Monitoring	Metrics Collection	Wall Time clock, Request Counter.

Prerequisite: The attacker needs to query the encryption oracle **twice** to get two valid (Ciphertext, Tag) pairs generated with the same Nonce.

3. Measured Results

This attack is significantly faster and more "fatal" than the Padding Oracle attack, as it relies on direct mathematical solving rather than brute-forcing 128 bytes.

Parameter	Measured Value	Notes
Oracle Requests	4 - 6 Requests	2 for initial samples, 1 for target ciphertext, 1 - 3 for final decryption.
Wall Time	~ 0.2203 seconds	SageMath solves the polynomial roots almost instantly.
Computational Cost	Extremely Low	Requires solving a low-degree polynomial, trivial for modern CPUs.
Success Rate	100%	The math is deterministic; if nonces are reused, the key is always recoverable.

4. Patching and Retesting

We modified the server code to adhere to cryptographic standards and re-ran the attack script.

Unique Nonce Generation

- **Change:** Modified server to generate a fresh random nonce inside the `encrypt()` function call, rather than defining it globally.
- **Result:**
 - Every encryption produced a unique Nonce (and thus a unique J_0).
 - The term $E_K(J_0)$ became a variable unknown for every equation.
 - The algebraic system became unsolvable (N equations with N+1 unknowns).

Attack Failed: The SageMath script could not find common roots or recover H.

5. Recommendations and Defensive Guidance

Nonce reuse in GCM is known as the "forbidden attack" because a single mistake compromises the integrity of the entire session (and in some cases, confidentiality via XOR keystream recovery).

1. **Always Rotate Nonces:** Ensure a **unique IV/Nonce** is generated for **every single encryption operation**. Never hardcode nonces or use static variables for them.
2. **Use 96-bit Nonces:** Standard AES-GCM is optimized for 96-bit (12-byte) nonces. Using other sizes triggers an extra GHASH step, which can be slower and more complex to implement correctly.
3. **Counter-Based Nonces:** If random generation is risky (e.g., low entropy in embedded devices), use a **deterministic counter** or a combination of Fixed Field + Counter, ensuring the counter never rolls over for the same key.
4. **Key Rotation:** If the maximum number of invocations for a single key (typically 2^{32} for random nonces to avoid Birthday Paradox collisions) is approached, rotate the encryption key immediately.

6. References

- Wikipedia - Galois/Counter Mode
(https://en.wikipedia.org/wiki/Galois/Counter_Mode)
- Hanno Böck, Aaron Zauner, Sean Devlin, Juraj Somorovsky, and Philipp Jovanovic,
Nonce-Disrespecting Adversaries: Practical Forgery Attacks on GCM in TLS
(<https://eprint.iacr.org/2016/475.pdf>)
- AES-GCM and breaking it on nonce reuse (https://frereit.de/aes_gcm/)